

Towards generic defensive cyber agents: Leveraging large language models and RAG-based few-shot learning

Muhammad Omer Farooq 

HSR Innovations and Consulting Cork, Ireland

ARTICLE INFO

Keywords:

Autonomous cyber defense
Cyber security
Large language models (LLMs)
Retriever augmented generation (RAG)
Few-shot learning

ABSTRACT

The dynamic and evolving nature of cyber threats requires defensive mechanisms that can adapt to new adversaries and network configurations without the high cost of retraining. This work introduces a generic cyber defense framework that integrates reinforcement learning (RL), large language models (LLMs), retrieval-augmented generation (RAG), and few-shot learning to achieve scalable and adaptive defense. The framework leverages a datastore of observation–action tuples collected from RL agents trained against specific adversaries. At inference, RAG retrieves the top- K relevant tuples for the current observation, which are incorporated into an LLM prompt to enable few-shot reasoning. The LLM then predicts the most suitable defensive action for the given observation, with adaptation to new adversarial behaviors and topology variations achieved by augmenting the datastore with additional observation–action tuples rather than retraining the generic defensive agent. Unlike state-of-the-art hierarchical or RL-supervised hybrid approaches, the proposed framework decouples adaptation from retraining of the deployed defender, enabling scalable incorporation of emerging threats. Evaluation across multiple adversarial scenarios and topology variants within the CybORG environment demonstrates that medium-sized models, particularly Mistral-7B and Gemma-7B, when combined with the proposed framework, deliver the most effective defensive actions. Comparative analysis further shows that our framework consistently outperforms existing methods in defensive success rates while enabling rapid extension to additional adversaries. These results establish the proposed approach as a scalable and practically deployable pathway for advancing autonomous cyber defense systems.

1. Introduction

The accelerating sophistication of cyber threats presents a persistent challenge to the design of defensive systems capable of reacting and adapting in real-time to adversaries employing novel strategies. Traditional rule-based intrusion detection and prevention mechanisms are inadequate in this context, as they rely on predefined signatures that adversaries can easily evade. Reinforcement Learning (RL) has emerged as a promising paradigm for autonomous cyber defense by enabling agents to learn adaptive policies through direct interaction with adversaries and dynamic environments [1]. However, conventional RL-based defensive agents typically lack scalability: when faced with new attack strategies or network topologies, these approaches require retraining the agent's policy from scratch, incurring significant computational cost and limiting practical deployment. To address this limitation, the proposed framework leverages previously collected observation–action tuples from RL agents as a reusable knowledge base. By augmenting this data store with tuples corresponding to new adversaries, the framework enables the generic defensive agent to select effective actions against previously unseen threats through few-shot reasoning with large language models, without retraining the agent itself.

Recent advances in large language models (LLMs) [2,3] and retrieval-augmented generation (RAG) [4] have opened new opportunities to develop adaptive cyber defense mechanisms. LLMs excel at reasoning over structured and unstructured data, while RAG enhances their contextualization by grounding predictions in relevant examples. Furthermore, few-shot prompting techniques allow LLMs to adapt to novel tasks from limited demonstrations, significantly reducing the data requirements for learning and retraining. These developments suggest a path toward building cyber defensive agents that combine the sequential decision-making power of RL with the generalization capabilities of LLMs.

In this work, we present a novel framework for designing generic cyber defensive agents that integrates RL traces, RAG, and LLM-based few-shot learning. The core idea is to maintain a data store of observation–action tuples collected from RL agents trained against specific adversaries. At inference time, RAG retrieves the top- K most relevant tuples given the current observation, which are then incorporated into an LLM prompt. This enables the LLM to predict the optimal defensive action based on both the current context and relevant past experience. Importantly, adaptability to new adversaries and

E-mail address: omer.farooq@gmail.com.

<https://doi.org/10.1016/j.comnet.2026.112349>

Received 14 December 2025; Received in revised form 11 April 2026; Accepted 29 April 2026

Available online 9 May 2026

1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

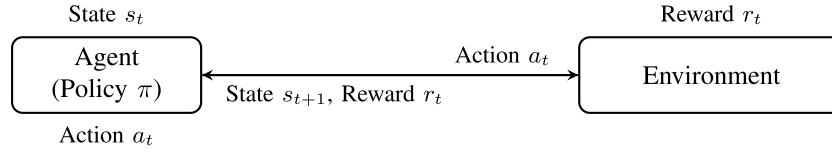


Fig. 1. Basic reinforcement learning framework. The agent selects actions based on observed states and receives feedback in the form of rewards from the environment.

network topologies is achieved by simply augmenting the data store with additional observation–action tuples, rather than retraining a new agent. This design represents a fundamental departure from existing state-of-the-art methods — including hierarchical defenses [5,6] and RL-supervised hybrids [7] — that require costly retraining to remain effective against evolving threats.

A comprehensive experimental evaluation demonstrates the efficacy of the proposed framework. Multiple LLM backbones of varying sizes were integrated into the framework, with results showing that medium-scale models such as *Mistral-7B* and *Gemma-7B* achieve the best performance. Comparative analysis against state-of-the-art methods confirms that our approach not only delivers superior defensive success rates but also uniquely enables rapid scaling to new adversaries and topologies without additional training.

The key contributions of this work are as follows:

- Novel integration of RL, RAG, and LLMs into a unified framework for cyber defense, enabling few-shot adaptation based on RL traces.
- Elimination of retraining overhead, with new adversarial scenarios handled via data store augmentation rather than agent retraining.
- Comprehensive evaluation of multiple LLMs in the proposed framework, demonstrating that *Mistral-7B* and *Gemma-7B* provide the most effective defensive actions.
- Demonstrated superiority over state-of-the-art baselines, including hierarchical methods and RL-supervised learning hybrids, in terms of both adaptability and defensive success rates.

The remainder of this paper is organized as follows. Section 2 presents background and related work. Section 3 details the design of the proposed framework. Section 4 presents the experimental setup and evaluation results. Section 5 discusses conclusions and potential future research directions.

2. Background and related work

2.1. Reinforcement learning

Reinforcement Learning (RL) [8] is a paradigm where an agent learns optimal behavior by interacting with an environment and receiving feedback in the form of rewards. As shown in Fig. 1, an RL setup consists of the agent, environment, state space, action space, and reward signal. At each step, the agent observes the current state s_t , selects an action a_t via policy $\pi(a|s)$, and receives a reward r_t as the environment transitions to s_{t+1} . Over repeated interactions, the agent refines its policy to maximize cumulative reward.

Unlike supervised learning, RL does not rely on labeled data but instead requires an interactive environment, making it well-suited for sequential decision-making tasks such as cyber defense. Here, agents learn through trial-and-error to detect, counter, and mitigate adversarial activity.

Deep Reinforcement Learning (DRL) extends RL by using deep neural networks as function approximators for policies or value functions. This allows agents to handle high-dimensional, partially observable environments and to generalize across dynamic, complex scenarios—capabilities particularly valuable in cyber defense.

2.1.1. Reinforcement learning formulation and optimization

We model the defensive agent’s task as an episodic Markov Decision Process (MDP), with the objective of maximizing cumulative reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^T r_t \right],$$

where r_t is the reward at time t and T the episode length. The environment is stochastic and partially observable, reflecting the uncertainty typical of cyber defense.

Rewards, defined by the CyBORG framework [9], penalize delayed or ineffective responses while rewarding timely defensive actions, aligning agent learning with goals such as minimizing compromise and lateral movement. For optimization, we adopt Proximal Policy Optimization (PPO), a policy-gradient method well suited for high-dimensional, dynamic environments due to its stability and sample efficiency.

2.2. Large language models

Large Language Models (LLMs) are deep learning models designed to understand and generate complex patterns from large-scale data. Traditionally, they have been used for natural language processing tasks such as text generation, summarization, and translation. However, in this work, we leverage LLMs in the context of cyber security. These models can process structured and unstructured cyber threat data, reason over complex attack patterns, and assist in tasks such as intrusion detection, threat mitigation, and automated defense strategy planning [10]. The effectiveness of LLMs in this domain arises from their ability to model long-range dependencies and adapt to diverse, high-dimensional input spaces.

2.2.1. Large language models used in this study

2.2.1.1. Mistral-7B. Mistral-7B is a 7.3 billion parameter model that employs Grouped-Query Attention (GQA) and Sliding Window Attention (SWA) to improve inference efficiency and handle long sequences. It has demonstrated strong performance in reasoning tasks, making it suitable for analyzing complex cyber security scenarios [11].

2.2.1.2. Gemma-7B. Developed by Google DeepMind, Gemma-7B is a decoder-only model optimized for reasoning and instruction-following tasks. Its open-weight availability allows for adaptation to cyber security applications, such as threat response recommendation and attack scenario analysis [12].

2.2.1.3. Llama-2-7B. Meta’s Llama-2-7B model has been fine-tuned using supervised learning and RL with human feedback (RLHF). Its capabilities in structured reasoning and planning can be used to simulate attack-defense interactions and improve automated defense policies [13].

2.2.1.4. Phi-3B. Phi-3B is a 3.8 billion parameter model designed for high performance in reasoning, coding, and mathematical tasks. In cyber security contexts, it can assist with automated analysis of logs, threat detection patterns, and decision-making in uncertain environments [14].

2.2.1.5. DeepSeek-1.5B. DeepSeek-1.5B is a compact reasoning-focused model that excels in tasks requiring structured problem solving. Despite its smaller size, it may demonstrate strong performance in evaluating complex threat scenarios, such as multi-step intrusions and attack chains [15].

2.2.2. Fine-tuning large language models with few-shot learning

Few-shot fine-tuning adapts pre-trained LLMs to specialized tasks using only a small set of labeled examples. Rather than training from scratch, it leverages existing knowledge to generalize with minimal data, reducing reliance on large annotated datasets and enabling rapid domain adaptation. In practice, the model is provided with limited task-specific input–output pairs, which guide accurate predictions on similar unseen inputs. This approach has been applied across diverse areas, including language understanding, reasoning, and code generation.

In cyber security, few-shot fine-tuning supports tasks such as classifying attack events or extracting structured intelligence from logs. Han et al. [16], for example, showed that an LLM fine-tuned with only a handful of samples could accurately identify cyber attack events, demonstrating the potential to quickly adapt to emerging threats without extensive labeled data.

2.3. Retriever-augmented generation

Retriever-Augmented Generation (RAG) is a hybrid approach that combines information retrieval with LLM generation. Instead of relying solely on the model's internal knowledge, RAG dynamically retrieves relevant documents or knowledge from an external corpus and conditions the generation on this retrieved information. This enables more accurate, context-aware, and up-to-date outputs, particularly in domains where specialized or evolving knowledge is required.

In the context of cyber security, RAG allows LLMs to access real-time threat intelligence, vulnerability databases, and historical attack patterns. This facilitates more informed decision-making, such as suggesting appropriate mitigation strategies, analyzing complex attack scenarios, or providing actionable insights for incident response. By grounding the generation in retrieved, domain-specific knowledge, RAG enhances reliability and reduces hallucinations commonly observed in standard LLM outputs.

2.4. RL-based environments for training cyber defense agents

A variety of RL environments have been designed to support the development and evaluation of autonomous cyber defense agents, including platforms such as those described in [17,18], and [19]. In this work, we adopt the CybORG framework [20], which was originally released as part of the CAGE Challenge. The following subsections provide an overview of CybORG, highlighting its architecture, adversarial roles, and relevance for evaluating RL-based strategies.

2.4.1. CybORG: A research environment for autonomous cyber operations

CybORG is a simulation framework for Autonomous Cyber Operations (ACO), modeled after OpenAI Gym [21]. It provides standardized environments for training and evaluating RL agents in adversarial cyber tasks. A typical scenario models a network with multiple subnets, routers, servers, and user hosts (Fig. 2). Agents assume distinct roles: red (attacker), blue (defender), and green (benign user traffic), enabling the study of defense strategies under realistic adversarial conditions.

CybORG defines two attacker models: *redmeander*, which compromises hosts sequentially, and *b_line*, which moves directly toward critical systems. Both exploit the same techniques but differ in strategy. The defender inspects hosts, removing or restoring when malicious activity is found. Representative red and blue actions are listed in Tables 1 and 2.

The defender's observation space captures host- and system-level details, encoded into a 52-dimensional binary vector for RL algorithms,

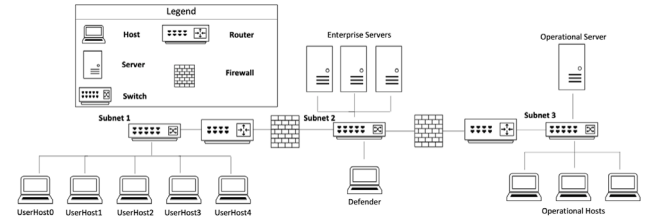


Fig. 2. Illustrative example of a network configuration within CybORG. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

invariant to the adversary. Red observations expand dynamically as compromise spreads. This wrapper ensures compatibility with toolkits like Stable Baselines and RLlib. Actions in CybORG are parameterized (e.g., host/IP input), and rewards are structured to reflect defense objectives. The CAGE Challenge 2 reward penalizes compromised hosts, critical servers, costly restores, and successful red impacts (Eq. (1), Tables 3, 4).

$$R_{\text{Blue}} = -0.1 \cdot (\text{host}_{\text{Red}} - 1) - \text{server}_{\text{Red}} - \text{Blue}_{\text{Restore}} - 10 \cdot (\text{Red}_{\text{Impact}}) \quad (1)$$

where:

- host_{Red} is the number of hosts with red root access,
- $\text{server}_{\text{Red}}$ is the number of critical servers with red root access,
- $\text{Blue}_{\text{Restore}} \in \{0, 1\}$ indicates whether the Blue agent executed a restore action,
- $\text{Red}_{\text{Impact}} \in \{0, 1\}$ indicates whether the red agent successfully impacted the operational server.

Finally, the red agent permanently controls User0, even after removal elsewhere. This persistent foothold reflects resilient attackers and underscores the need for proactive, adaptive defenses rather than one-time remediation.

2.5. Reinforcement learning approaches for defensive agent training

Existing research on defensive agents for ACOs generally falls into four broad categories [22]:

- Single-agent methods
- Hierarchical agent frameworks
- Ensemble-based strategies
- Hybrid approaches combining RL with supervised learning

In single-agent approaches, a single blue agent is trained to respond to a spectrum of red agent attacks, acting as the sole defender within the network [23]. Hierarchical methods adopt a multi-level structure in which a high-level agent delegates tasks to specialized sub-agents, each designed to counter specific categories of adversarial behavior [5]. Ensemble strategies instead rely on multiple agents trained independently. At runtime, these agents evaluate the system state and suggest defensive actions, after which the most frequently proposed action is executed. A detailed evaluation of ensemble-based defensive strategies has been provided in [22].

A different direction has focused on blending reinforcement learning with supervised machine learning. For instance, in [24], data generated from multiple RL-trained agents was used to train classifiers such as Support Vector Machines (SVM) and Multi-Layer Perceptrons (MLP). The resulting model demonstrated the feasibility of building a generalized defensive agent, though its evaluation was limited to the training network topology and conventional supervised methods.

Table 1
Representative actions for red agents.

Action	Description
Sleep	Idle action; no parameters required.
DiscoverRemoteSystems	Executes a ping sweep of a subnet to discover active IP addresses.
DiscoverNetworkServices	Performs a port scan on a host to enumerate open services.
ExploitRemoteService	Attempts to exploit a remote service to gain shell access. Requires target IP.
EternalBlue	Specific exploit targeting Windows systems to obtain SYSTEM privileges.
PrivilegeEscalate	Attempts privilege escalation from user-level to root/SYSTEM, provided shell access exists.
FTPDDirectoryTraversal	Explores restricted directories.
SQLInjection	Inserts malicious SQL code for database exploitation.
Impact	Disrupts a service; requires host name parameter.

Table 2
Representative actions for blue agent.

Action	Description
Sleep	Idle action; no parameters required.
Analyze	Scans a host for malware indicators. Requires host name as parameter.
Remove	Eliminates adversary's user-level shell access.
Restore	Reverts a host to a clean state, but incurs a large penalty due to operational disruption.
Deception	Deploys a decoy service to mislead the attacker.
Monitor	Collects host state information.

Table 3
Blue agent penalties for red administrative access.

Subnet	Hosts	Reward penalty
Subnet 1	User hosts	-0.1
Subnet 2	Enterprise servers	-1
Subnet 3	Operational server	-1
Subnet 3	Operational host	-0.1

Table 4
Reward penalties for successful adversarial actions.

Agent	Host	Action	Penalty
Red	Operational server	Impact	-10
Blue	Any	Restore	-1

2.6. Discussion

The challenge of constructing generalized defensive agents remains central to advancing ACO. While hierarchical RL and RL-supervised hybrids have demonstrated advantages over ensemble- and single-agent schemes, both hierarchical and ensemble-based approaches require deploying multiple agents concurrently. This introduces additional overhead and can delay decision-making due to coordination requirements. A more efficient and scalable alternative lies in developing a single adaptive agent that generalizes across adversaries and network topologies.

Achieving this goal requires methods capable of modeling sequential dependencies, as adversarial activity often unfolds over multiple steps. LLM-based system augmented with RAG-based few shot learning offer promising capabilities in this regard, enabling effective detection of complex, multi-stage attack campaigns. To facilitate this, our study leverages the CyBORG environment, a standardized platform well-suited for red-vs-blue training scenarios. CyBORG provides realistic and extensible network simulations that support consistent evaluation across varying topologies and adversary behaviors, aligning closely with the objectives of training adaptive and generalized defensive agents.

3. RAG-based generic cyber defense system

This section presents the proposed framework for designing generic defensive cyber agents by integrating RL, RAG, and LLMs. The framework is motivated by the need for autonomous defensive agents that

can adapt to diverse adversarial behaviors and evolving network topologies without requiring extensive retraining. In modern cyber environments, adversaries increasingly employ machine learning-driven attack strategies, which challenges traditional defenses that rely on static models or hand-crafted rules. Our approach addresses this challenge by combining the adaptability of RL with the generalizability of LLMs and the contextual reasoning offered by RAG-based few-shot learning.

The framework operates in two interconnected phases. In the first phase, RL-based defensive agents are trained against specific adversaries, with each agent learning strategies tailored to its opponent. Once trained, these agents are executed against their respective adversaries to generate observation-action tuples. These tuples capture the interaction dynamics between defenders and attackers and are subsequently stored in a knowledge base, forming a repository of diverse defensive experiences.

In the second phase, the RAG-based few-shot learning system leverages this knowledge base to enable real-time decision-making. For each time step, the current observation vector is provided to a retriever module, which employs cosine similarity to identify the top- K most relevant observation-action samples from the knowledge base. The retriever then augments the prompt with these contextual examples, along with task-specific instructions, and forwards the enriched prompt to an LLM. The LLM processes the augmented input and recommends an appropriate defensive action, effectively generalizing beyond the scenarios originally encountered during RL training. This mechanism allows the system to adapt to new adversarial behaviors or network topologies by simply expanding the knowledge base, without retraining the entire model.

The overall architecture of the proposed framework is illustrated in Fig. 3. The figure highlights the interaction between the RL component, the knowledge base of observation-action tuples, the RAG-based retriever, and the LLM-driven decision-making process. Each of these components is described in detail in the following subsections, with particular emphasis on their roles in enabling scalable, flexible, and effective defensive cyber operations.

3.1. Proposed methodology – Reinforcement learning component

3.1.1. Training reinforcement learning models against targeted adversaries

In the proposed methodology, each defensive agent is initially trained using RL to counter a designated adversary. Before commencing training, a systematic hyperparameter (HP) optimization process is performed for every agent, as careful tuning of HPs is known to significantly improve the stability and efficiency of learning in RL and machine learning more broadly. In this study, HP tuning is carried out within the CyBORG environment. The process involves evaluating multiple candidate configurations for key HPs of the chosen RL algorithm, i.e., PPO. The aim is to identify the settings that produce the most effective learning outcomes. This extends and generalizes the optimization strategy originally introduced in our earlier work [24].

The HP optimization procedure systematically explores a predefined search space for several critical parameters associated with PPO. These include, but are not limited to, batch size, learning rate, discount factor,

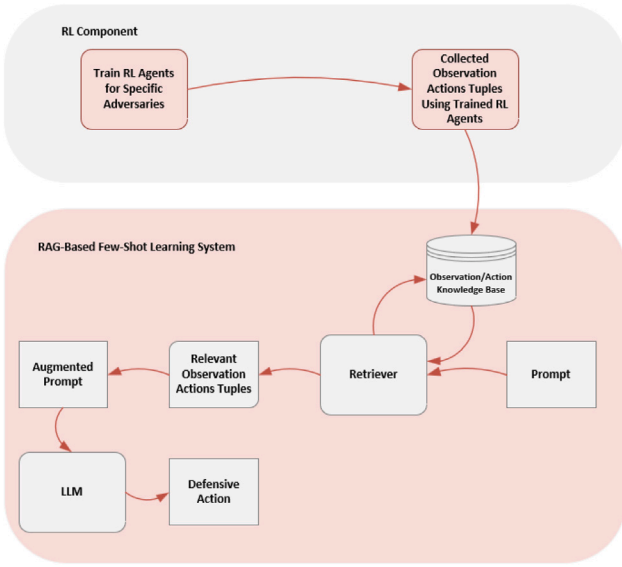


Fig. 3. System architecture for a generic defensive cyber agent. The RL component trains agents against specific adversaries to generate observation-action tuples, which populate the RAG-based knowledge base. The RAG-based few-shot learning system retrieves the most relevant samples using cosine similarity, augments the prompt with these examples, and passes it to an LLM that recommends defensive actions in real-time.

number of training steps, entropy coefficient, clipping range, number of epochs, GAE lambda, gradient clipping norm, value-function loss coefficient, activation functions, and neural network architecture. The tuning pipeline follows a structured sequence of steps:

- Develop an RL-based simulation scenario in the Cyborg environment, modeling various network entities (e.g. hosts, firewalls, servers, routers, and switches) together with a defensive agent and its adversary.
- Specify the ranges of candidate values for all relevant HPs.
- Define the search space and employ an automated HP optimization framework, such as Optuna [25], to efficiently navigate this space.
- Record the HP configuration that achieves the best performance for each defensive agent when tuned against its assigned attacker.
- Repeat this process independently for each defensive agent to ensure adversary-specific optimization.

Once the optimal set of HPs is identified, each defensive agent is trained in its adversarial environment using the selected PPO configuration. This modular and adversary-specific training strategy enables every defensive model to be tailored to the unique behaviors and tactics of its opponent. The precise ranges explored during the tuning, along with the final configurations selected for each agent, are reported in Section 4.

3.1.2. Building the knowledge base: Collecting observation-action tuples

In the Cyborg training environment, the defensive agent operates within a discrete action space of 153 actions, including host-specific and global operations such as *Sleep*, *Monitor*, *Analyze*, *Remove*, *Restore*, and various *Misinform* actions (decoy strategies). Global actions such as *Sleep* and *Monitor* apply at the network level, while host-specific actions are tied to individual hosts; the same action applied to two different hosts is represented by distinct action identifiers. The observation space encodes system- and host-level indicators (e.g., process state, user activity, and service status), which are transformed by Cyborg's wrapper

into a fixed-length binary vector of size 52. This compact representation makes observation-action interactions amenable for downstream processing.

To support the development of a generic defensive agent, our framework incorporates a structured pipeline to collect observation-action tuples that reflect defensive decision making against multiple adversaries. Initially, specialized defensive agents are trained with RL against specific red agents, as described previously. Each of these agents adapts to the tactics and conditions induced by its corresponding adversary, thereby learning adversary-specific policies. To capture these behaviors in a reusable form, we execute each trained RL agent against its adversary over multiple episodes, during which the environment records the resulting observation-action pairs. These tuples encapsulate how defensive strategies unfold in various attack scenarios and constitute the foundation of the knowledge base for our RAG-based few-shot learning mechanism.

To ensure coverage across diverse adversarial strategies, this procedure is repeated for all N specialized RL agents. The resulting tuples are aggregated into a unified repository which serves as the knowledge base from which relevant examples are later retrieved during inference. By shuffling and consolidating data across agents, we avoid sequential clustering that might bias the system toward the behavior of any single defender. This step is critical to preserve diversity in the knowledge base, thereby improving the effectiveness of retrieval during few-shot prompting. Instead of directly training a supervised model, as in prior work, the knowledge base here plays a central role in augmenting LLM prompts with representative defensive samples. This design allows the system to scale naturally: as new adversaries or network topologies are encountered, additional episodes can be simulated to expand the repository without requiring retraining of the full defensive model.

Although the joint observation-action space may appear large, our framework maintains scalability by relying on fixed-size binary observation vectors (52 dimensions) paired with discrete action labels (from 153 possible actions). By systematically collecting, diversifying, and organizing this data, we construct a knowledge base tailored to support retrieval-augmented LLM reasoning, enabling the defensive agent to generalize across adversarial strategies in real-time.

3.2. RAG-based few-shot learning and inference

In this section, we describe RAG-based few-shot learning and inference framework employed to guide the defensive agent in selecting optimal actions based on observed environmental states. This approach combines historical knowledge retrieval with LLM reasoning to facilitate accurate decision-making in high-dimensional observation spaces.

The process begins with the generation of an *input prompt*, which encodes the current observation vector and instructions for the LLM to classify it into a discrete defensive action. This prompt establishes the task for the LLM, specifying the output format and the range of valid actions, while preparing the model to utilize relevant examples for few-shot reasoning. The careful design of this prompt ensures that the LLM focuses solely on selecting the correct action without producing extraneous explanations or reasoning.

Once the input prompt is prepared, it is passed to the *retriever module*, which identifies the most relevant historical observation-action tuples from a pre-existing knowledge base. The retriever measures the similarity between the current observation vector and the stored vectors using *cosine similarity*, a widely adopted metric for high-dimensional feature spaces. Cosine similarity quantifies the angular distance between two vectors and is defined as shown in Eq. (2), where $\mathbf{A}$ and $\mathbf{B}$ are n -dimensional vectors representing the current and stored observations, respectively. Higher cosine similarity values indicate closer alignment between the vectors, allowing the retriever to select the top- K most similar observation-action tuples that serve as few-shot examples for the LLM.

$$\text{cosine_similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2)$$

The retrieved top-K examples are then appended to the original input prompt to create an *augmented prompt*. By presenting these examples alongside the current observation, the LLM can identify patterns and relationships between features and actions, enabling it to make informed predictions even when encountering observations that differ from previously seen states. This few-shot learning paradigm effectively leverages both the agent's historical knowledge and the reasoning capacity of the LLM. The *LLM inference module* receives the augmented prompt and analyzes both the current observation and the retrieved examples to output a discrete defensive action. The LLM selects an integer action from the specified range that best matches the observed features based on the patterns present in the examples. This process is repeated iteratively each time the defensive agent encounters a new observation, ensuring that decisions are made in real-time while adapting to the evolving environment.

By combining historical knowledge retrieval with LLM reasoning, this architecture provides a flexible and adaptive defense mechanism against previously unseen attacks. The use of top-K retrieval ensures that the LLM has access to contextually relevant examples, while the few-shot reasoning capability allows it to generalize from these examples to novel observations. As a result, the defensive agent can dynamically respond to new attack patterns without requiring retraining in all possible scenarios, enabling robust and scalable defense in complex environments.

3.2.1. Input prompt

The complete input prompt used for LLM few-shot reasoning is shown below:

You are an expert classifier. Your only task is to output a single integer action at a time

Do not output any explanation, reasoning, or text. Just output the final integer action (e.g., 23).

Your goal is to select the most appropriate action (an integer between 0 and 151) for a new 52-bit binary feature vector.

To help you decide, you will be shown examples of binary features and the corresponding actions taken. You must select the action that best matches the new features based on the shown examples only.

Each example is in the following format:
Features: <52-bit binary string>
Action: <integer between 0 and 151>

Use the patterns in the examples to determine the best match for the new observation.

Do NOT invent new actions -- only select from the actions shown in the examples.

=== EXAMPLES START ===
EXAMPLES RETRIEVED BY THE RETRIEVER ARE APPENDED HERE
=== EXAMPLES END ===

The prompt used in our framework is intentionally constrained to produce a single integer action without explanation. This design choice reflects the nature of the task, which is structured decision classification over a fixed 52-bit feature vector. Unlike open-ended reasoning tasks, the objective here is deterministic action selection rather than

natural language generation. Allowing free-form reasoning (e.g., chain-of-thought prompting) could introduce verbosity, ambiguity, or invalid action outputs, requiring additional parsing and validation. By enforcing direct classification, we ensure stable and reproducible outputs that align with the discrete action space of the cyber defense environment. The prompt is therefore designed to emulate structured decision-making rather than natural language reasoning. Also note that chain-of-thought prompting was not employed because the task does not require multi-step reasoning in natural language. Preliminary exploratory experiments indicated that allowing intermediate reasoning did not improve action accuracy and occasionally introduced formatting inconsistencies. Therefore, we adopted a constrained output formulation to ensure reliability and comparability across models.

3.3. Augmenting the datastore for new adversaries

When a previously unseen adversary is introduced, the proposed framework is extended by appending new observation-action tuples to the retrieval datastore. These tuples are obtained by training an RL agent (PPO in our implementation) against the new adversary in the simulation environment and recording its interaction trajectories. The resulting tuples are directly incorporated into the datastore used by the generic defensive agent without modifying its parameters. Generating tuples for a new adversary involves two stages. First, PPO HPs are selected via short training runs of 50,000 environment steps per candidate configuration. After selecting the best configuration, a final PPO agent is trained. At least 10,000 observation-action tuples are collected from trajectories collected after the trained agent interacts with the adversary. These tuples are then indexed into the datastore and immediately available to the defensive agent via retrieval-augmented prompting.

Although this process requires training an adversary-specific RL agent, it differs fundamentally from conventional approaches that retrain a full defensive policy for each new threat. In the proposed framework, the generic defensive agent and its LLM remain unchanged and continuously operational; adaptation is achieved purely through datastore augmentation. Consequently, extending to a new adversary incurs only the one-time cost of PPO training and tuple extraction, while avoiding retraining, redeployment, or downtime of the defensive agent. This enables scalable and non-disruptive incorporation of emerging threats.

4. Performance evaluation

4.1. Evaluation with multiple LLMs

The number of retrieved observation-action tuples (K) directly influences the contextual information available to the LLM during action generation. To assess its impact, we varied K from 1 to 7 and evaluated defensive success rate across representative adversarial scenarios. Performance improved as K increased from 1 to 3, indicating that providing multiple relevant exemplars enhances the LLM's ability to infer appropriate defensive actions. For $K > 3$, performance plateaued and slightly decreased. This behavior is consistent with retrieval-augmented prompting dynamics: small K provides insufficient contextual diversity, whereas large K can introduce less relevant or conflicting tuples, increasing prompt noise and reducing decision clarity. The best overall performance was obtained at $K = 3$, and this value is therefore used in all reported experiments.

The first stage of evaluation integrates the proposed framework with five LLMs of varying sizes and architectures: Mistral-7B, Gemma-7B, LLaMA2-7B, Phi-3B, and DeepSeek-1.5B. This comparison serves two purposes: (i) to identify the most effective model for guiding defensive agents, and (ii) to examine how model scale and architecture affect performance in this domain. Larger models (Mistral-7B, Gemma-7B, LLaMA2-7B) are expected to capture richer contextual dependencies,

while smaller models (Phi-3B, DeepSeek-1.5B) test whether compact architectures can still provide effective decision support. Although not the primary focus, we note that larger models require greater computational resources, highlighting trade-offs between performance and deployment efficiency. All evaluated LLMs are instruction-tuned variants, which are better aligned with structured instruction-following tasks.

This design enables us to assess whether parameter count directly correlates with defensive performance, or whether architectural factors allow smaller models to remain competitive. Evaluating a diverse set of LLMs ensures the framework is not tied to a single backbone and can adapt to environments with different hardware constraints. In summary, this stage functions as model selection: identifying the best-performing LLMs for later comparison against state-of-the-art baselines, while providing insights into the relationship between scale, architecture, and decision quality in cyber defense tasks.

4.1.1. Training RL agents for red agents in Cyborg

As the proposed framework requires an observation-action knowledge base, we first train defensive RL agents against the two adversarial agents in Cyborg, *b_line* and *red_meander*, which represent distinct attacker strategies. Separate training ensures the knowledge base captures diverse observation-action trajectories. Training is conducted on the network topology shown in Fig. 2, providing the environment structure for both attacker and defender.

RL training is highly sensitive to hyperparameters (HPs) such as learning rate, batch size, discount factor, and entropy coefficient, which strongly affect stability and performance. To address this, we employ Optuna [25] for systematic HP optimization. Logarithmic scales are used for sensitive parameters (e.g., learning rate, entropy coefficient), while broader ranges are explored for batch size and training steps. For each red agent, 30 Optuna trials were conducted, where each trial corresponded to a complete 50,000 steps PPO training run with periodic evaluations after 5000 steps. The optimization process was performed independently for each adversary. The best-performing configuration, determined by highest average episodic reward, was selected for final training. We observed convergence of performance improvements after approximately 20–25 trials, suggesting that additional trials yielded diminishing returns. Although the HPs optimization budget of 30 trials ($\approx 1.5M$ environment steps per adversary) is non-trivial, this cost is incurred only once per adversary during offline PPO training and is not part of the online operation of the proposed defensive framework. We selected Optuna rather than exhaustive grid search because the PPO search space includes continuous, logarithmic, and categorical HPs, making grid-based exploration computationally prohibitive. Compared with unguided random search, Optuna provides a more sample-efficient adaptive optimization strategy under a fixed trial budget. The resulting one-time tuning cost is therefore a practical tradeoff for obtaining stronger adversary-specific policies from which high-quality observation-action tuples can be extracted for datastore augmentation. Table 5 summarizes the search ranges and final HPs for both red agents.

We adopt the Cyborg-defined reward function, consistent with prior work, to ensure comparability with state-of-the-art baselines and avoid introducing framework-specific biases. This alignment allows for transparent benchmarking of defensive performance across methodologies.

4.1.2. Network topologies

To assess the scalability and adaptability of the proposed framework, we evaluate it on three network topologies. Topology I corresponds to the training environment, while Topologies II and III introduce structural variations to test generalization. All topologies share the same number of nodes, subnet-based segmentation, and comparable activity levels, ensuring that performance differences stem primarily from structural changes.

Table 5

Explored ranges and optimal hyperparameters for RL-based defensive agents.

Hyperparameter	Explored interval	Optimal (b_line)	Optimal (red_meander)
Batch size	8–512	8	256
No. of steps	8–2048	128	1024
Discount factor (γ)	0.9–0.9999	0.90	0.95
Learning rate	$[10^{-5}, 1]$ (log scale)	1.89×10^{-4}	3.73×10^{-3}
Entropy coefficient	$[10^{-8}, 0.1]$ (log scale)	1.03×10^{-5}	1.76×10^{-2}
Clip range	0.1–0.4	0.3	0.3
Epochs per update	1–20	10	10
GAE λ	0.8–1.0	0.8	0.8
Max gradient Norm	0.3–5.0	0.5	5.0
Value function Coeff.	0–1	0.0919	0.7358
Activation function	tanh, relu, elu, leaky_relu	ReLU	tanh
Network architecture	{64,64}, {256,256}	{256,256}	{64,64}

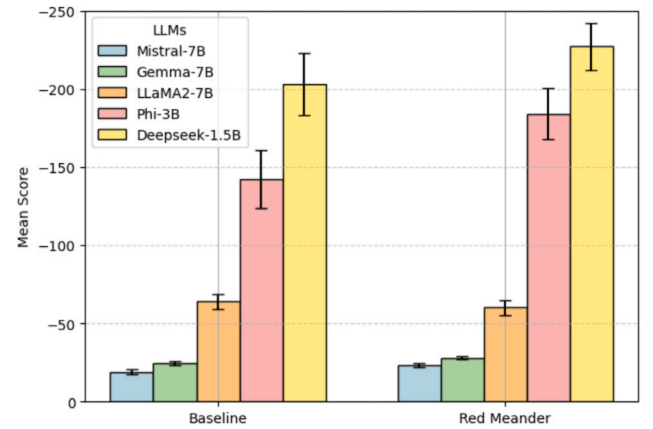


Fig. 4. Performance comparison using network Topology I. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

- **Topology I (Training Topology)** – Baseline configuration used for training RL agents and extracting observation-action trajectories. It consists of three subnets; UserHost0 in Subnet 1 is permanently compromised as the initial threat vector (Fig. 2).
- **Topology II** – Two hosts are moved from Subnet 1 to Subnet 2, while the compromised host remains in Subnet 1. This alters subnet cardinalities and benign traffic distribution without changing the attack origin, testing resistance to overfitting (Fig. 5).
- **Topology III** – The compromised host is relocated from Subnet 1 to Subnet 2, altering both subnet sizes and the entry point of malicious activity. This creates a new threat surface and lateral movement opportunities, testing adaptability under reconfigured conditions (Fig. 7).

These variations mirror realistic network dynamics such as device relocation or subnet restructuring. Comparing performance across them highlights the framework's capacity to generalize beyond the training environment and remain robust under evolving deployment conditions.

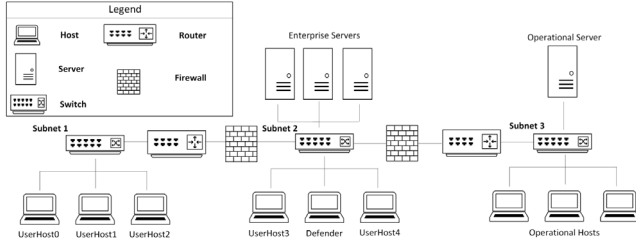
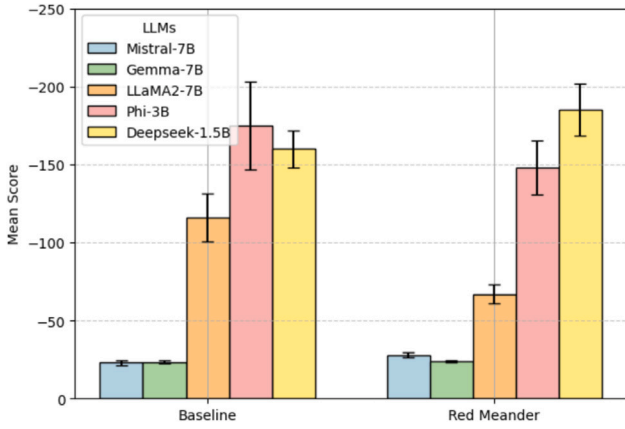
4.1.3. Training-Evaluation separation and Topology shift

The datastore used for retrieval-augmented inference was constructed exclusively from trajectories generated by PPO agents trained in Topology I. No observation-action tuples from Topologies II or III were included in the knowledge base. Evaluation results for Topologies II and III therefore represent adaptation to topology variations that were not directly observed during RL-based tuple generation. Furthermore, evaluation episodes are independent from the PPO training episodes used during hyperparameter optimization and tuple collection. Only the collected observation-action pairs were retained in

Table 6

Total mean reward — (Topology I).

Agent type	Mistral	Gemma	Llama2	Phi	DeepSeek
Total mean reward	−42	−52.5	−124	−326	−430

**Fig. 5.** Network Topology II.**Fig. 6.** Performance comparison using network Topology II.

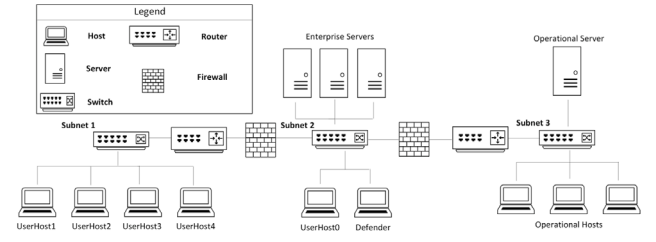
the datastore, while all reported performance metrics are computed on separate evaluation runs. Although Topologies II and III share the same node identities as Topology I, structural modifications — such as node relocation and compromised host reassignment — alter attack paths, observation patterns, and defensive decision contexts. Consequently, evaluation on these topologies reflects adaptation under modified structural conditions rather than replay of previously observed trajectories.

4.1.4. Evaluation of LLM integration on training Topology

We first evaluated the framework on the training topology (Topology I). Each episode consisted of 50 steps, repeated 100 times, with results reported as mean total rewards and 95% confidence intervals (Fig. 4). The overall mean rewards against both red agents are shown in Table 6.

Action usage analysis (Table 7) highlights key differences. The best-performing models, Mistral-7B and Gemma-7B, showed balanced use of Analysis, Remove, and Restore, alongside dominant but not exclusive reliance on Deception. In contrast, weaker models such as DeepSeek-1.5B and Phi-3B relied heavily on Deception while unnecessarily invoking Sleep or Monitor, indicating poor contextual understanding.

Overall, larger models (Mistral-7B, Gemma-7B) achieved the highest rewards, with Mistral-7B leading. Smaller models (Phi-3B, DeepSeek-1.5B) performed poorly, showing limited adaptability. Interestingly, LLaMA2-7B underperformed relative to models of similar scale, suggesting that architecture and training data quality also strongly influence defensive effectiveness.

**Fig. 7.** Network Topology III.

4.1.5. Evaluation of LLM integration on Topology II

We next evaluated the framework on Topology II, which modifies subnet composition while retaining the overall structure. Each experiment again consisted of 100 runs of 50 steps, with results reported as mean total rewards and 95% confidence intervals (Fig. 6, Table 8). On this topology, Gemma-7B achieved the best performance (−47.5), slightly surpassing Mistral-7B (−51), indicating stronger adaptability to the altered subnet configuration. LLaMA2-7B continued to under perform, and smaller models (Phi-3B, DeepSeek-1.5B) again lagged substantially, with larger confidence intervals reflecting instability across runs. Action distributions (Table 9) further clarify these trends. Stronger models balanced their use of Analysis, Remove, and Restore alongside dominant Deception. Notably, Gemma-7B employed Remove more frequently than Mistral-7B, which may explain its edge here. In contrast, weaker models misallocated actions to Sleep and Monitor, reflecting poor contextual interpretation.

Comparing Topologies I and II reveals three patterns. First, while Mistral-7B remained strong, Gemma-7B overtook it in this setting, showing that architecture and training, not parameter count alone, drive robustness. Second, smaller models consistently performed poorly across both topologies, constrained by limited action diversity. Third, greater use of Analysis and Remove correlated with higher rewards, underscoring the role of action diversity in adapting to network changes. Overall, Topology II reinforces earlier findings: larger LLMs enable richer, context-aware defense, whereas compact models fail to generalize. The shift in ranking between Mistral-7B and Gemma-7B highlights the importance of balancing scale with adaptability in dynamic environments.

4.1.6. Evaluation of LLM integration on Topology III

We next evaluated the framework on Topology III, which relocates the compromised host to a different subnet, altering both subnet sizes and the entry point of attacks. This tests adaptability to structural shifts. The setup mirrors earlier experiments (100 runs, 50 steps each), with results reported as mean total rewards and 95% confidence intervals (Fig. 8), aggregated rewards (Table 10), and action usage (Table 11). Gemma-7B and Mistral-7B again performed best (−39.5 and −50), with Gemma-7B leading across both adversaries. LLaMA2-7B showed moderate results (−120), while Phi-3B (−186) and DeepSeek-1.5B (−302) lagged substantially, underscoring the difficulty smaller models face in generalizing under complex conditions.

Action statistics confirm deception as the dominant defense (>70% across models). Gemma-7B distinguished itself with higher use of restore actions (16.4% vs. *baseline*, 6.6% vs. *red meander*), suggesting more balanced strategies. Phi-3B leaned heavily on remove (17.2%), reflecting an aggressive but less effective approach, while DeepSeek-1.5B again wasted actions on sleep and monitor, explaining its weak performance.

Comparisons across topologies show two consistent patterns: (i) Gemma-7B and Mistral-7B remain the most reliable, with Gemma-7B overtaking Mistral-7B in Topologies II and III; and (ii) weaker models fare slightly better here than in Topology II (e.g., Phi-3B improved from −323 to −186, LLaMA2-7B from −183 to −120), suggesting exploitable

Table 7
Defensive agents action usage (%) — Topology I.

Action	Baseline					Red meander				
	Mistral	Gemma	Llama2	Phi	DeepSeek	Mistral	Gemma	Llama2	Phi	DeepSeek
Sleep	0	0	0	0	10.6	0	0	0	0	9.4
Monitor	0	0	0	0	10	0	0	1	0	15.2
Analysis	19.5	17	17.8	18	3.8	18	17	14.4	13.4	3
Remove	5.7	7	6.2	3	4.2	10.2	8	5.2	3.2	2.6
Restore	1.2	2.5	0.6	2.6	0.4	1.8	3	1.2	3.4	0.8
Deception	73.6	73.5	75	82	71	70	72	78.2	80	69

Table 8
Total mean reward — (Topology II).

Agent type	Mistral	Gemma	Llama2	Phi	DeepSeek
Total mean reward	−51	−47.5	−183	−323	−345

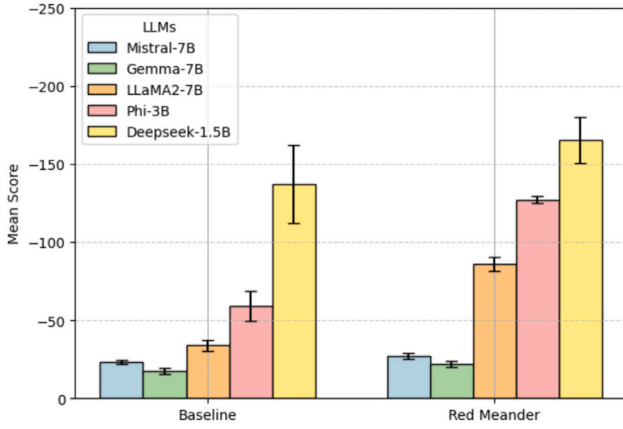


Fig. 8. Performance comparison using network Topology III.

structural cues. DeepSeek-1.5B, however, remained consistently poor. In summary, Topology III confirms earlier findings: larger models (especially Gemma-7B) adapt best to shifting environments, while deception remains the backbone defensive strategy across all topologies.

4.2. Comparison with state-of-the-art defensive agent training methods

We benchmark the proposed framework against three categories of state-of-the-art defensive agents. First, hierarchical RL agents (HA5) [5], where a high-level agent identifies the red agent (with 5% recognition error) before invoking the corresponding defender. Second, RL-based approaches from [7], which collect observation–action tuples and train generic defenders using supervised methods: support vector machines (RL-SVM), multilayer perceptrons (RL-MLP), and XGBoost (RL-XGBoost). Third, specialized agents trained exclusively against a single adversary (*b_line* or *red_meander*), denoted RL and RED, which often perform well but lack generalizability.

All methods are evaluated in the CybORG environment under identical conditions and reward functions for fairness. This setup allows direct comparison of our LLM-based RAG framework against both generalized and adversary-specific baselines. Each experiment consisted of 100 runs of 50 steps per episode, with results reported as mean total rewards and 95% confidence intervals.

4.2.1. Performance comparison — Topology I

Fig. 9 and Table 12 summarize the performance of the proposed LLM-based framework against existing state-of-the-art defensive agent training methods under Topology I. Each experiment consisted of 100 runs with an episode length of 50 steps, and results are reported as mean total rewards with 95% confidence intervals.

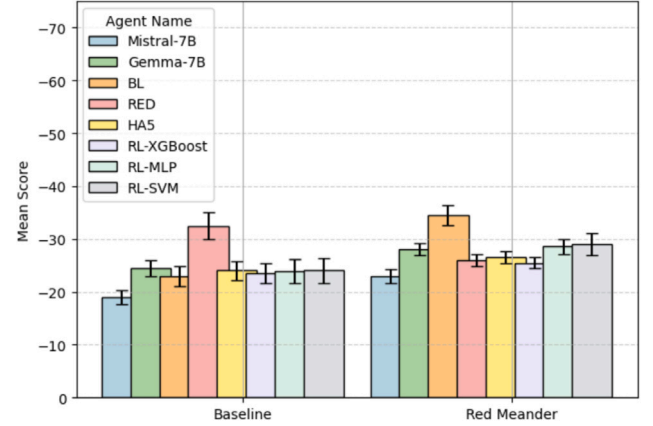


Fig. 9. Performance comparison with existing methods — Topology I.

Among all evaluated methods, Mistral-7B achieved the strongest performance with a total mean reward of −42, followed by Gemma-7B (−52.5). Both LLM-based agents outperformed hierarchical RL (HA5), supervised-learning-based approaches (RL-SVM, RL-MLP, RL-XGBoost), and specialized agents trained exclusively against a single red adversary (BL and RED). Notably, specialized agents demonstrated strong results against their targeted adversary but failed to generalize across both *b_line* and *red_meander*, leading to overall weaker mean scores. Similarly, supervised-learning-based methods achieved moderate results but required the additional step of training a generic model from collected observation–action tuples, introducing scalability limitations.

In contrast, the LLM-based framework does not require retraining for new adversaries. By leveraging retrieval-augmented generation, the system adapts directly from stored observation–action trajectories, enabling efficient handling of previously unseen red agents. This property provides a clear scalability advantage: while prior methods must invest additional training effort to construct generic defensive models, the proposed approach requires only the collection of new tuples, thereby reducing overhead and accelerating deployment.

Overall, the results on Topology I demonstrate that larger LLMs (Mistral-7B and Gemma-7B) not only deliver superior mean rewards but also achieve this without adversary-specific retraining, offering both effectiveness and adaptability in dynamic cyber defense environments.

4.2.2. Performance comparison — Topology II

Fig. 10 presents the mean reward along with 95% confidence intervals and Table 13 present the total mean rewards for different defensive methods under Topology II respectively. As with Topology I, each experiment was run for 100 episodes of length 50.

In this configuration, Gemma-7B achieved the best overall performance with a total mean reward of −47.5, followed closely by Mistral-7B at −51. Both LLM-based agents again surpassed RL baselines such as RL-XGBoost (−52.5), RL-MLP (−57), and RL-SVM (−57). Among the classical baselines, HA5 performed reasonably well (−53), but specialized red-focused agents (BL and RED) fared the worst with rewards of −61 and −60.5, respectively. This ranking is consistent with the

Table 9
Defensive agents action usage (%) — Topology II.

Action	Baseline					RedMeander				
	Mistral	Gemma	Llama2	Phi	DeepSeek	Mistral	Gemma	Llama2	Phi	DeepSeek
Sleep	0	0	0	0	10	0	0	0	0	10.2
Monitor	0	0	0	0	14.6	0	0	0	0	15.8
Analysis	21.4	18	10.8	14.2	3.4	16	15	19.6	16	3.4
Remove	9.8	16	8.2	6	5	8.9	6.2	9.2	6	3.2
Restore	2.8	2	1.4	3.4	0.4	3.4	3.4	1.4	3.4	0.6
Deception	66	64	79.6	76.4	66.6	71.7	75.4	69.8	74.6	66.8

Table 10
Total mean reward — (Topology III).

Agent type	Mistral	Gemma	Llama2	Phi	DeepSeek
Total mean reward	−50	−39.5	−120	−186	−302

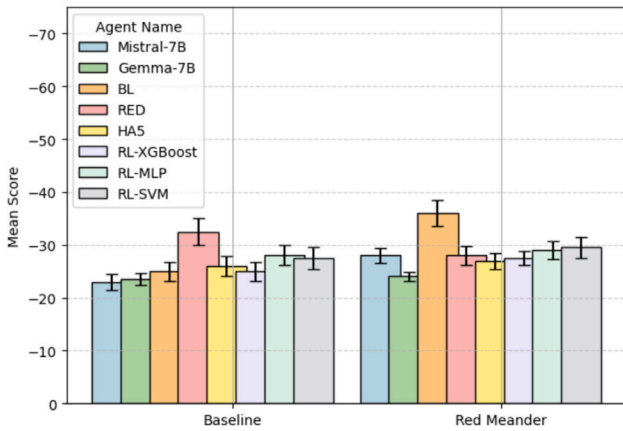


Fig. 10. Performance comparison with existing methods — Topology II.

findings from Topology I, where LLM-based agents also outperformed both RL and specialized agents.

When comparing across the two topologies, several trends emerge. First, Gemma-7B demonstrates consistently strong performance: while it trailed Mistral-7B in Topology I (−52.5 vs. −42), it overtakes all methods in Topology II with −47.5. This indicates that Gemma-7B adapts more effectively to the structural changes introduced in Topology II, showcasing its flexibility in reasoning about different network layouts. Conversely, Mistral-7B, which was the top performer in Topology I, remains highly competitive in Topology II but shows slightly reduced relative performance. Together, these results highlight the complementary strengths of the two LLMs, both of which generalize well without the need for retraining.

In contrast, traditional RL methods remain less scalable across topologies. While their scores remain clustered in the mid−50s, they consistently trail behind LLM-based approaches, reflecting their dependency on topology-specific training. The poor performance of BL and RED across both topologies reinforces this observation: these agents are highly tuned to particular adversarial strategies and fail to generalize when faced with new red agents or altered network structures.

Overall, comparing Topology I and II confirms that agents trained through the proposed methodology offer robust adaptability and improved defensive capability across diverse adversarial conditions. Their ability to maintain leading performance without additional model training underscores the scalability advantage of the proposed framework relative to existing state-of-the-art methods.

4.2.3. Performance comparison — Topology III

Fig. 11 reports the mean rewards of defensive agents against individual red agents including 95% confidence intervals. Table 14 complements this by presenting the total mean rewards aggregated across

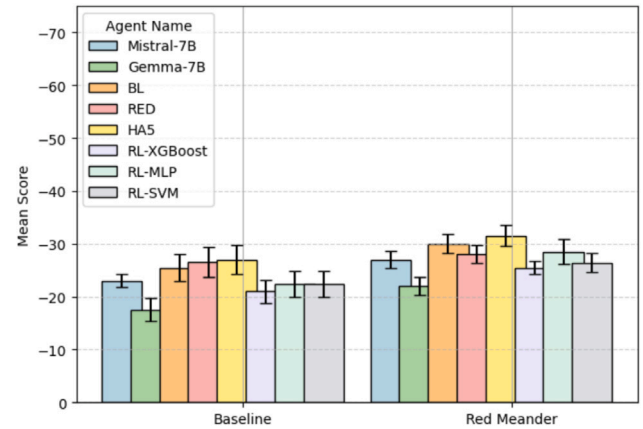


Fig. 11. Performance comparison with existing methods — network Topology III.

both red agents. Each experiment was run for 100 episodes of length 50 to ensure statistical reliability. In this setting, Gemma-7B achieved the best overall performance with a total mean reward of −39.5, marking the strongest result across all three topologies. Mistral-7B also performed competitively with a reward of −50, outperforming most RL-based baselines. Among RL methods, RL-XGBoost was the strongest (−46.5), although still behind Gemma-7B. RL-MLP (−50.9) and RL-SVM (−48.8) produced results close to the Mistral-7B, while HA5 agent deteriorated further with −58.5. As observed in earlier topologies, BL and RED again performed poorly (−55.5 and −54.5, respectively), reflecting their lack of generalizability.

When comparing across topologies, clear trends emerge. In Topology I, Mistral-7B was the strongest agent (−42), while in Topology II, Gemma-7B led with −47.5. In Topology III, Gemma-7B not only outperforms Mistral-7B again but also establishes the best overall result (−39.5). This demonstrates that the agents based on the proposed methodology not only generalize across environments but also adapt differently depending on the network topology and adversary behavior. In contrast, RL-based baselines show limited adaptability. Although RL-XGBoost remains competitive in Topology III, other RL variants fluctuate in effectiveness, and specialized agents such as BL and RED consistently underperform. These results highlight the reliance of traditional approaches on adversary-specific training and their lack of scalability.

Across all three topologies, the proposed framework consistently delivers superior or competitive defensive performance without re-training. Results presented here highlight that the agents using the proposed framework maintain stable effectiveness across individual red agents, while the tables confirm strong aggregate performance. Notably, Gemma-7B in Topology III (−39.5) achieves the best overall defensive outcome, underscoring the scalability and robustness of LLM-driven approaches for autonomous cyber defense.

Table 11
Defensive agents action usage (%) — Topology III.

Action	Baseline					Red meander				
	Mistral	Gemma	Llama2	Phi	DeepSeek	Mistral	Gemma	Llama2	Phi	DeepSeek
Sleep	0	0	0	0	11.8	0	0	0	0	9.8
Monitor	0	0	0	0	10.2	0	0	0	0	11.6
Analysis	15	7	9.8	9.4	2	15	16	11.2	11.4	2.8
Remove	9.7	4.6	7.6	17.2	2.8	11	13.8	5.2	8.8	3.6
Restore	3.3	16.4	0.2	2.6	0.4	5.7	6.6	0.8	4.6	1
Deception	72	72	82	70.8	72.8	73	63.6	82.4	75.2	71.2

Table 12
Total mean reward comparison with existing methods — (Topology I).

Agent type	Mistral	Gemma	BL	RED	HA5	RL-XGBoost	RL-MLP	RL-SVM
Total mean reward	−42	−52.5	−57.5	−58.5	−50.5	−49.0	−52.39	−53

Table 13
Total mean reward comparison with existing methods — (Topology II).

Agent type	Mistral	Gemma	BL	RED	HA5	RL-XGBoost	RL-MLP	RL-SVM
Total mean reward	−51	−47.5	−61	−60.5	−53	−52.5	−57	−57

Table 14
Total mean reward comparison with existing methods — (Topology III).

Agent type	Mistral	Gemma	BL	RED	HA5	RL-XGBoost	RL-MLP	RL-SVM
Total mean reward	−50	−39.5	−55.5	−54.5	−58.5	−46.5	−50.9	−48.8

4.3. Prevalence of deception actions

Across all evaluated models, adversaries, and network topologies, deception actions constitute the majority of defensive responses (typically 65–80%). This consistent pattern arises from both environmental and algorithmic factors. First, in the considered cyber-defense environment, deception actions provide an effective means of disrupting adversary progression while incurring minimal operational risk. Unlike removal or restoration actions, which require high confidence and may impact legitimate services, deception can be applied under uncertainty and remains reversible. Consequently, RL defenders trained in this environment frequently learn deception as a safe and broadly applicable mitigation strategy. Second, because the proposed framework relies on retrieval of observation–action tuples generated by RL agents, the action distribution in the datastore reflects these learned policies. As deception actions are prevalent in successful RL trajectories, they are more frequently retrieved and presented as few-shot exemplars to the LLM, reinforcing their selection during inference. This effect is consistent across LLM architectures.

Finally, deception actions are structurally less dependent on precise state diagnosis than actions such as removal or restoration, which require accurate identification of compromised assets. Under partial observability, this makes deception a robust default defensive response, further increasing its usage frequency. These observations indicate that the dominance of deception actions is not a model artifact but a consequence of both the cyber-defense dynamics of the environment and the experience distribution captured in the RL-derived datastore.

4.4. Discussion on generalization scope

While the proposed framework is designed to enable adaptation to new adversaries and network configurations through datastore augmentation rather than retraining, the empirical evaluation in this work is conducted within the CybORG environment using two adversary types and three topology variants. The topology variations involve structural modifications such as node relocation and compromised host reassignment, which alter the defensive context but do not constitute fundamentally different network paradigms. Therefore, the presented results demonstrate intra-environment generalization, i.e., adaptation

across adversarial behaviors and structural variations within a consistent simulation framework. Although these variations meaningfully change the defensive decision space, they do not exhaustively represent all possible network architectures or real-world cyber environments. Author further note that the topology variations considered in this work (Topologies II and III) primarily involve structural perturbations of Topology I, such as node relocation and compromised host reassignment, while preserving the overall network composition and node types. As such, these scenarios do not represent fundamentally different network architectures (e.g., variations in subnet organization, node heterogeneity, or security zoning). Therefore, the current evaluation should be interpreted as demonstrating adaptation under controlled topology variations within a fixed environment, rather than full generalization across diverse network architectures.

Importantly, the architectural contribution of the proposed framework—decoupling adaptation from policy retraining via retrieval-augmented inference is environment-agnostic. In principle, the same mechanism can be applied to broader network settings, provided that representative observation–action tuples are available. Evaluating cross-simulator and real-world deployment scenarios constitutes an important direction for future work.

4.5. Inference latency and computational requirements

Since autonomous cyber defense systems require timely decision-making, we evaluated the computational requirements of the proposed framework during inference. All experiments were conducted locally on a MacBook equipped with an Apple Silicon processor, integrated GPU acceleration via Apple’s Metal framework, and 32 GB of unified memory. The evaluated large language models were executed using Ollama backend. The prompt used by the defensive agent is intentionally compact, consisting of a short instruction template, a small number of retrieved observation–action examples, and a 52-bit binary feature vector representing the current observation. Furthermore, the model generates only a single token corresponding to the selected action identifier. This constrained output format significantly reduces generation overhead compared to typical natural language generation tasks.

Table 15
Approximate inference latency per decision and per episode (500 steps).

Model	Parameters	Latency per decision (ms)	Time per episode (s)
Mistral-7B	7B	504	252
Gemma-7B	7B	396	198
LLaMA2-7B	7B	576	288
Phi-3B	3B	252	126
DeepSeek-1.5B	1.5B	144	72

Table 15 summarizes the approximate inference latency per defensive decision for each evaluated LLM. To provide conservative estimates, the reported values correspond to high-end measurements. Each episode in the evaluation environment consists of 500 decision steps; therefore, the table also reports the approximate inference time required to complete a full episode. Even for the largest models evaluated in this study, the inference latency per decision remains within 500 ms, which is compatible with step-based decision cycles in the CyBORG cyber defense environment. Since defensive decisions occur once per simulation step, the measured latency is well within the operational requirements for near real-time automated response. It is also worth noting that further reductions in latency can be achieved through standard deployment optimizations such as model quantization, batching, and optimized inference runtimes. Additionally, smaller models such as Phi-3B and DeepSeek-1.5B already demonstrate substantially lower inference times, providing viable options for deployments where computational resources are constrained.

In practical deployments, transient inference delays may occur due to system load or resource contention. However, the proposed framework operates in a step-based decision setting rather than strict real-time packet-level processing, which provides tolerance to moderate latency fluctuations. In addition, several standard mitigation strategies can be employed, including asynchronous execution, caching of recent retrieval results, and fallback to lightweight models or predefined safe actions under high-load conditions. These mechanisms ensure that occasional inference spikes do not significantly impact overall defensive effectiveness. Author also clarify that the reported latency values include the complete decision pipeline, encompassing retrieval of the top- K observation–action tuples from the datastore, prompt construction, and LLM inference. In our implementation, the retrieval step incurs negligible overhead compared to LLM inference due to the relatively small datastore size and efficient similarity search. Therefore, the reported latency is dominated by the model inference stage.

5. Conclusion and future work

This work introduced a scalable and adaptive framework for autonomous cyber defense that integrates RL, LLMs, RAG, and few-shot learning. Unlike prior approaches — such as hierarchical agents or RL-supervised hybrids — the proposed framework does not require retraining of the deployed generic defensive agent when new adversaries or topology variations are introduced. Moreover, it does not rely on additional supervised training to construct a general defensive policy. Instead, adaptability is achieved by augmenting the knowledge store with new observation–action tuples, which the LLM leverages through retrieval-based prompting. This design ensures scalability and rapid extensibility, addressing a fundamental limitation of existing state-of-the-art methods. Experiments across three network topologies and two adversarial agents (*b_line* and *red_meander*) demonstrated the effectiveness of the framework within the CyBORG environment. The results showed that medium-sized LLMs — particularly Mistral-7B and Gemma-7B — consistently produced the strongest defensive performance. Gemma-7B emerged as the most reliable across topology variations, highlighting the importance of model adaptability in addition to scale. Comparative evaluation further revealed that

the proposed approach outperformed hierarchical and RL-supervised baselines and matched or exceeded specialized agents trained exclusively against individual red agents. Importantly, this was achieved without retraining the deployed defender, underscoring the practicality and efficiency of the method. The analysis of defensive action distributions further confirmed that stronger models exhibited richer and more balanced strategies, while weaker baselines often misallocated actions to less effective responses. Overall, the findings indicate that retrieval-augmented prompting provides a promising pathway for enabling adaptable and flexible cyber defense agents. By coupling the structured experience of RL-trained agents with the reasoning capabilities of LLMs, the framework bridges the gap between specialized training and extensible defense. This demonstrated ability to adapt across unseen adversarial behaviors and topology variations within the evaluated environment represents a meaningful advancement over existing methods.

While the proposed framework demonstrates strong adaptability and scalability within the evaluated setting, further investigation is warranted to assess performance across more diverse adversaries and fundamentally different network environments. Future work will extend evaluation to larger-scale and more dynamic scenarios, such as those modeled in the CAGE Challenge 4, where scenarios vary at each reset and involve multi-agent defense. Such evaluation will provide deeper insight into cross-environment robustness and scalability. In conclusion, this work establishes that combining RL experience with LLM reasoning through retrieval-augmented prompting enables the design of scalable and adaptive defensive agents. By avoiding the retraining burden of prior methods for the deployed defender, the framework offers a practical pathway for advancing autonomous cyber defense systems in evolving adversarial landscapes.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

References

- [1] T.T. Nguyen, V.J. Reddi, Deep reinforcement learning for cyber security, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (8) (2023) 3779–3795.
- [2] T.B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D.M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: *Proceedings of the 34th International Conference on Neural Information Processing Systems*, 2020.
- [3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: Open and efficient foundation language models, 2023, *ArXiv, abs/2302.13971*. [Online]. Available: <https://api.semanticscholar.org/CorpusID:257219404>.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Proceedings of the 34th International Conference on Neural Information Processing Systems*, 2020.
- [5] M. Foley, C. Hicks, K. Highnam, V. Mavroudis, Autonomous network defence using reinforcement learning, in: *ASIA CCS' 22, Association for Computing Machinery*, New York, NY, USA, 2022, pp. 1252–1254.
- [6] A.V. Singh, E. Rathbun, E. Graham, L. Oakley, S. Boboila, A. Oprea, P. Chin, Hierarchical multi-agent reinforcement learning for cyber network defense, 2025, [Online]. Available: <https://arxiv.org/abs/2410.17351>.
- [7] M.O. Farooq, T. Kunz, Combining supervised and reinforcement learning to build a generic defensive cyber agent, *J. Cybersecur. Priv.* 5 (2) (2025).

- [8] M. Naeem, S.T.H. Rizvi, A. Coronato, A gentle introduction to reinforcement learning and its application in different fields, *IEEE Access* 8 (2020) 209320–209344.
- [9] Cage challenge 2, 2024, <https://github.com/cage-challenge/cage-challenge-2>, Last accessed 27 July 2024.
- [10] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, D. Meng, When LLMs meet cybersecurity: a systematic literature review, *Cybersecurity* 8 (2025) 55.
- [11] A.Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D.S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L.R. Lavaud, M.-A. Lachaux, P. Stock, T.L. Scao, T. Lavril, T. Wang, T. Lacroix, W.E. Sayed, Mistral 7B, 2023, [Online]. Available: <https://arxiv.org/abs/2310.06825>.
- [12] H. Face, Google/Gemma-7b, 2024, [Online]. Available: <https://huggingface.co/google/gemma-7b>.
- [13] H. Touvron, et al., Llama 2: Open foundation and fine-tuned chat models, 2023, arXiv preprint [arXiv:2307.09288](https://arxiv.org/abs/2307.09288). [Online]. Available: <https://arxiv.org/abs/2307.09288>.
- [14] M. Abdin, et al., Phi-3 technical report: A highly capable language model, 2024, arXiv preprint [arXiv:2404.14219](https://arxiv.org/abs/2404.14219). [Online]. Available: <https://arxiv.org/abs/2404.14219>.
- [15] Medium, DeepSeek-R1-Distill-Qwen-1.5B: The best small-sized LLM?, 2025, [Online]. Available: <https://medium.com/data-science-in-your-pocket/deepseek-r1-distill-qwen-1-5b-the-best-small-sized-llm-14eee304d94b>.
- [16] Y. Han, et al., Cyberattack event and arguments extraction based on few-shot learning, *Sci. Rep.* 15 (1) (2025).
- [17] A. Molina-Markham, C. Minitier, B. Powell, A. Ridley, Network environment design for autonomous cyberdefense, 2021, CoRR, [abs/2103.07583](https://arxiv.org/abs/2103.07583). [Online]. Available: <https://arxiv.org/abs/2103.07583>.
- [18] L. Li, J.-P.S.E. Rami, A. Taylor, J.H. Rao, T. Kunz, Unified emulation-simulation training environment for autonomous cyber agents, 2023, [Online]. Available: <https://arxiv.org/abs/2304.01244>.
- [19] Cyber battle sim, 2025, <https://github.com/microsoft/CyberBattleSim>, Last accessed 20 August 2025.
- [20] C. Baillie, M. Standen, J. Schwartz, M. Docking, D. Bowman, J. Kim, CybORG: An autonomous cyber operations research gym, 2020.
- [21] OpenAI gym, 2025, (<https://gymnasium.farama.org>), Last accessed 25 August 2025.
- [22] M. Kiely, D. Bowman, M. Standen, C. Moir, On autonomous agents in a cyber defence environment, 2023, [Online]. Available: <https://arxiv.org/abs/2309.07388>.
- [23] T. Kunz, C. Fisher, J.L. Novara-Gsell, C. Nguyen, L. Li, A multiagent cyberbat-tlesim for RL cyber operation agents, 2023, [Online]. Available: <https://arxiv.org/abs/2304.11052>.
- [24] M.O. Farooq, T. Kunz, A generic blue agent training framework for autonomous cyber operations, in: 2024 IFIP Networking Conference, IFIP Networking, 2024, pp. 515–521.
- [25] Optuna: A hyperparameter optimization framework, 2025, (<https://optuna.readthedocs.io/en/stable/>), Last accessed 7 August 2025.